

**MDMC runs for 1000 steps
(Argon tutorial)**

Calculation set-ups: parameters which were the same in all calculations

```
# Imports used for this tutorial
import numpy as np
import os
from scipy.interpolate import interp2d
from MDMC.control import Control
from MDMC.MD import Atom, Dispersion, LennardJones, Simulation, Universe
# Change the number of threads depending on the number of physical cores on your computer
# as it was tested for LAMMPS
os.environ["OMP_NUM_THREADS"] = "4"
# Build universe with density 0.0176 atoms per AA^-3
density = 0.0176
# This means cubic universe of side:
# 23.0668 A will contain 216 Ar atoms
# 26.911 A will contain 343 Ar atoms
# 30.7553 A will contain 512 Ar atoms
# 38.4441 A will contain 1000 Ar atoms
universe = Universe(dimensions=20.75)
Ar = Atom('Ar', charge=0.)
# Calculating number of Ar atoms needed to obtain density
n_ar_atoms = int(density * np.product(universe.dimensions))
print(f'Number of argon atoms = {n_ar_atoms}')
universe.fill(Ar, num_struc_units=(n_ar_atoms))
Ar_dispersion = Dispersion(universe,
                           (Ar.atom_type, Ar.atom_type),
                           cutoff=8.,
                           function=LennardJones(epsilon=1.0243, sigma=3.36))

# MD Engine setup
simulation = Simulation(universe,
                       engine="lammcs",
                       time_step=1.9848602234,
                       temperature=120.,
                       traj_step=77)

# Energy Minimization and equilibration
simulation.minimize(n_steps=5000)
simulation.run(n_steps=30000, equilibration=True)
# exp_datasets is a list of dictionaries with one dictionary per experimental
# dataset
# Dataset from: van Well et al. (1985). Physical Review A, 31(5), 3391-3414
# resolution is None as the original author already accounted for instrument resolution
exp_datasets = [{'file_name': 'Well_s_q_omega_Ar_data.xml',
                  'type': 'SQw',
                  'reader': 'xml_SQw',
                  'weight': 1.,
                  'auto_scale': True,
                  'resolution': None}]
fit_parameters = universe.parameters
```

Physically reasonable time-step, which can also fulfill the correct number of output frames

Calculation set-ups: parameters which were varying in all calculations

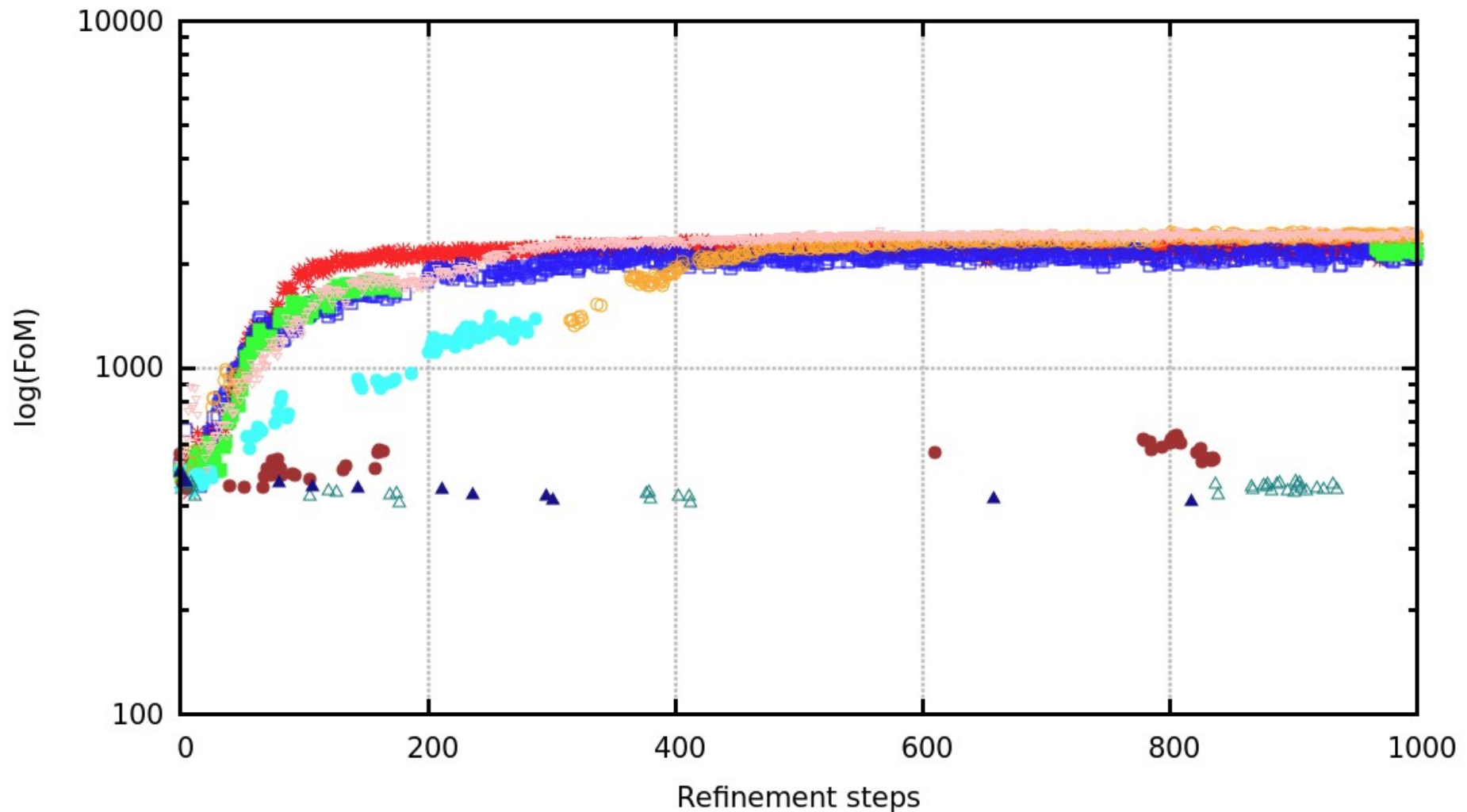
```
control = Control(simulation=simulation,  
                  exp_datasets=exp_datasets,  
                  fit_parameters=fit_parameters,  
                  MC_norm=20,  
                  minimizer_type="MMC",  
                  reset_config=True,  
                  MD_steps=4000,  
                  max_parameter_change=0.1)  
# Run the refinement, i.e. refine the FF parameters against the data  
control.refine(n_steps=150000)
```

MC_norm was different for each calculation

max_parameter_change was equal to 0.05 in every calculation

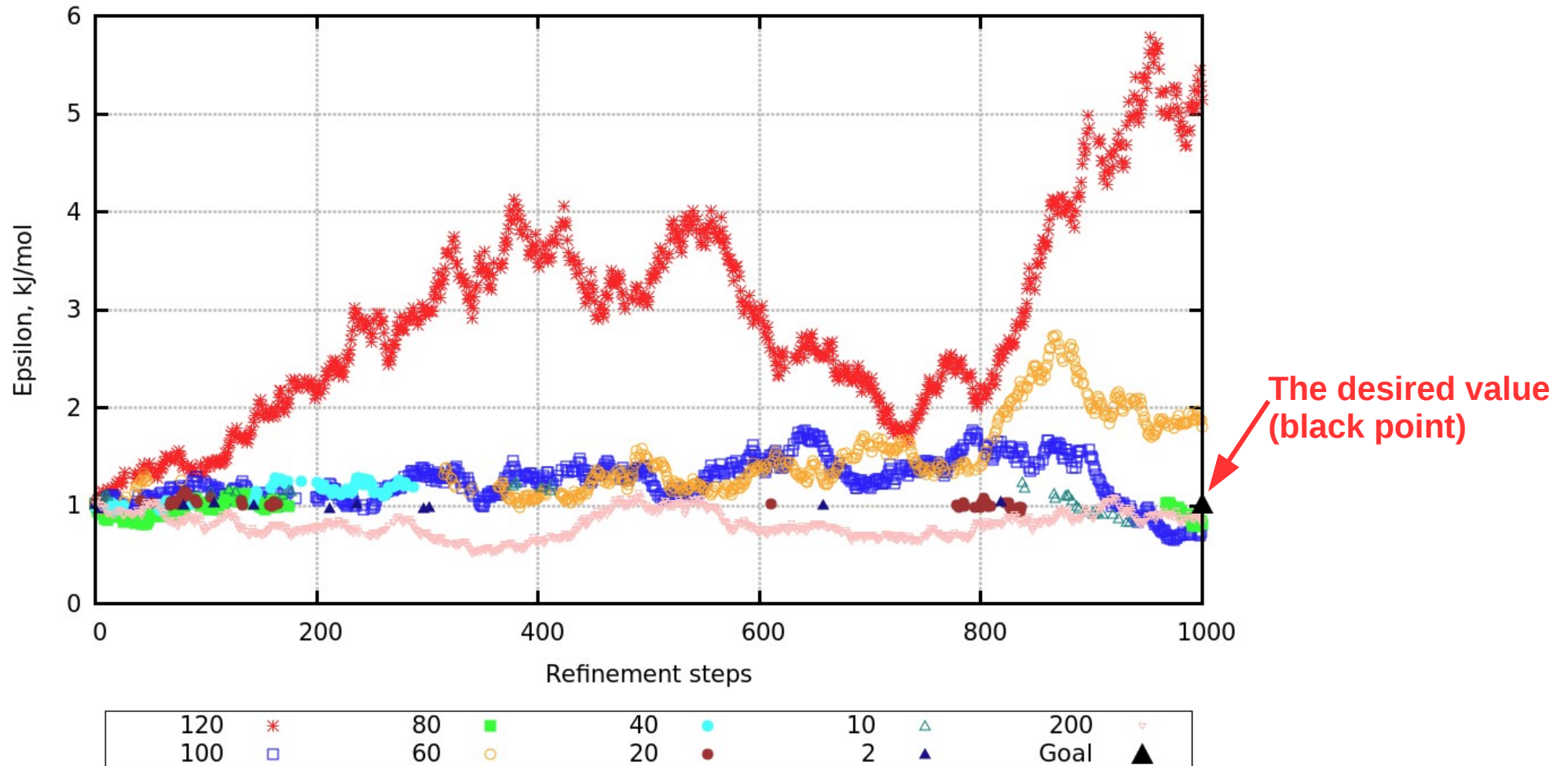
The number of steps was equal to 1000

Results: Figure of merit over the time (plotted only for accepted values)

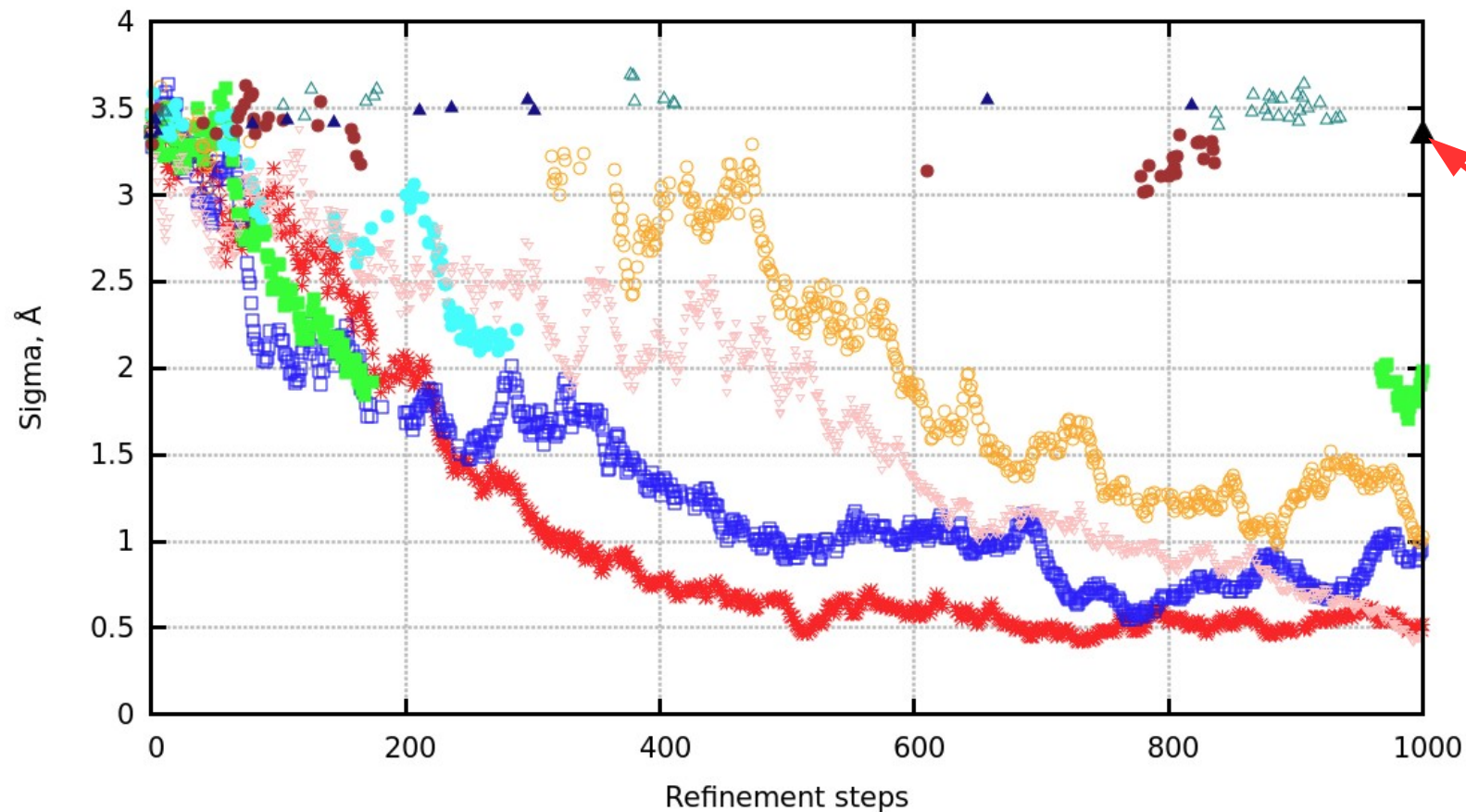


120	*	80	■	40	●	10	△	200	▽
100	□	60	○	20	●	2	▲		

Results: Accepted values of epsilon over the time



Results: Accepted values of sigma over the time



The desired value
(black point)